CASE STUDY

Architecting a Ride-Hailing & Delivery Super-App

Companion exercise to the "Software Architect Role" lecture

How to Use This Document

1. Read the Scenario Briefing and Requirements carefully.
2. Work through the 6 tasks in order, writing your own answers. Use a notebook, whiteboard, or blank doc.

Part 1- Scenario Briefing

You have just been hired as the **Lead Software Architect** for **Zippo**, a fast-growing ride-hailing and food-delivery "super-app" operating in South and Southeast Asia. Zippo currently runs as a single monolithic application on one cloud region and has hit a wall.

Business context

- Zippo has grown from 50,000 to 5 million monthly active riders and 300,000 active drivers across 6 countries in 18 months.
- The board has approved expansion into 10 more countries within a year, including markets with unreliable mobile networks.
- Investors are pressing for a food/grocery delivery vertical to launch within the same platform, reusing the driver fleet.
- A recent regional promotion caused a 3-hour outage during peak surge, drivers could not receive trip offers, riders were double-charged, and the company lost an estimated $400,000 plus significant reputational damage.
- The CEO has told you plainly: "I don't care how you fix it, but this cannot happen again, and it has to work in villages with 2G internet, not just capital cities."

Current (broken) system, for context

- One large monolithic backend, one PostgreSQL database, deployed in a single AWS region.
- Driver location updates are written directly to the same database rows read by the matching logic under load, these read/write locks are the main bottleneck.
- Payments are processed synchronously inside the same request that ends a trip; if the payment gateway is slow, the whole trip-completion flow hangs.
- There is no message queue, all communication is direct HTTP calls between internal modules.

Your job: **propose a target architecture** that fixes these problems and supports the company's growth plans.

Part 2 - Requirements

Functional requirements

- F1. Riders can request a ride or place a food order and see nearby available drivers/couriers in real time.
- F2. The system matches a rider/order to the best available driver within a target time.
- F3. Drivers receive trip/order offers and can accept/reject within a short countdown.
- F4. Riders and drivers can track each other's live location during an active trip.
- F5. Payment is captured at trip completion, supporting cards, wallets, and cash.
- F6. Pricing must support surge pricing based on real-time supply/demand.
- F7. The platform must support both ride-hailing and food delivery using a shared driver/courier pool.

Non-functional requirements (quality attributes) — with numbers

Attribute	Target
Matching latency	Rider sees a matched driver within 3 seconds for 95% of requests
Location update throughput	300,000 drivers pinging location every 4 seconds ≈ 75,000 writes/sec at steady state, up to 3x during surge
Availability	99.95% uptime for trip-critical services (matching, payments)
Payment consistency	Zero double-charges or lost charges, financial data must be strongly consistent
Geographic scale	Must operate across 16 countries, some with strict data-residency laws (payment data must stay in-country)
Network tolerance	Driver apps must function acceptably on 2G/3G connections with intermittent connectivity
Extensibility	Adding a new vertical (e.g., grocery delivery) should not require rewriting the matching or payment core
Observability	Any trip's full lifecycle must be traceable end-to-end for dispute resolution

Constraints

- Budget allows for cloud infrastructure but not unlimited , cost-awareness matters, especially in price-sensitive markets.

Part 3 - Your Tasks

Work through these in order. Write down your actual answers , bullet points are fine ,before expanding the solution for each.

- Task 1 — Prioritize quality attributes. Given the numbers above, which 3 quality attributes are most in tension with each other? Which would you prioritize, and what would you deliberately sacrifice?
- Task 2 — Decompose the system. Draw or list the major services/components you would create. What is each one responsible for, and what should NOT cross a service boundary?
- Task 3 — Design the real-time matching flow. Walk through, step by step, what happens from the moment a rider taps “Request Ride” to the moment a driver is matched. Where do writes happen? Where do reads happen? What happens if no driver is found in 3 seconds?
- Task 4 — Solve the data consistency problem. Location data and payment data have very different consistency needs. Propose a data strategy that treats them differently and explain why.
- Task 5 — Design for the outage that already happened. Specifically address: (a) surge-time overload, (b) payment gateway slowness blocking trip completion, (c) the 2G-connectivity requirement.
- Task 6 — Identify top 3 risks. What could still go wrong with your proposed architecture, and how would you mitigate each?